

— CCAF CERTIFICATION SERIES

Introduction to Model Context Protocol

Anthropic 官方认证课程完整笔记

14节课精华整理 + Python SDK 实战 + 官方文档补充

organized by Syneira

2026.06

MCP 是什么

Model Context Protocol (MCP) 是一个开源标准，用于连接 AI 应用和外部系统。就像 USB-C 为电子设备提供统一接口，MCP 为 AI 应用提供统一的外部系统连接方式。

"MCP is an open-source standard for connecting AI applications to external systems. Think of MCP like a USB-C port for AI applications."

MCP 解决什么问题

假设你要让 Claude 访问 GitHub 数据。没有 MCP，你需要自己编写所有 GitHub 功能的 tool schema 和调用函数，涵盖 repositories、pull requests、issues、projects 等海量功能。

"Without MCP, you'd need to create an incredible number of tool schemas and functions to handle all of GitHub's features. That's a lot of effort and ongoing maintenance burden."

MCP 的解决方案：把工具定义和执行的负担从你的服务器转移到专门的 MCP Server。GitHub MCP Server 已经封装好所有功能，你只需连接即可。

MCP 的能力

- Agent 可以访问你的 Google Calendar 和 Notion，充当个性化助手
- Claude Code 可以根据 Figma 设计生成完整 Web 应用
- 企业聊天机器人可以连接多个数据库，让用户通过对话分析数据
- AI 模型可以在 Blender 中创建 3D 设计并通过 3D 打印机输出

MCP 与 Tool Use 的区别

常见误解：MCP 和 Tool Use 是同一件事

MCP Server 提供已经定义好的工具 schema 和实现。Tool Use 是 Claude 实际调用这些工具的机制。两者互补但不同，关键区别在于**谁做了实现工作**：MCP 意味着别人已经帮你实现了。

"MCP servers provide tool schemas and functions already defined for you, while tool use is about how Claude actually calls those tools. The key difference is who does the work."

生态支持

MCP 是开放协议，支持广泛的客户端和服务端。Claude、ChatGPT、VS Code、Cursor 等都支持 MCP，实现了 build once, integrate everywhere。

MCP 架构

MCP 架构由三个角色组成：Host、Client、Server。理解它们的关系是掌握 MCP 的基础。

三个角色

角色	说明	示例
MCP Host	AI 应用本身	Claude Desktop, VS Code, IDE
MCP Client	Host 内部的通信桥梁，每个 Server 对应一个	SDK 提供的 ClientSession
MCP Server	暴露工具/资源/提示词的程序	GitHub Server, 文件系统 Server

"The MCP client serves as the communication bridge between your server and MCP servers. It's your access point to all the tools that an MCP server provides."

传输协议

MCP 是 **transport agnostic**（传输无关）的，客户端和服务端可以通过不同协议通信：

协议	适用场景	机制
stdio	本地，同一台机器	客户端启动服务器作为子进程，通过 stdin/stdout 通信，换行分隔 JSON-RPC
Streamable HTTP	远程连接	单个 MCP endpoint，POST 发请求，GET 开 SSE 流接收服务器推送

Streamable HTTP 详细机制（官方文档补充）

- Session 通过 MCP-Session-Id header 管理
- SSE 支持 Last-Event-ID 断线重连
- 协议版本 header：MCP-Protocol-Version: 2025-11-25
- 安全：必须验证 Origin header 防止 DNS rebinding 攻击
- 向后兼容旧版 HTTP+SSE transport (2024-11-05)

消息类型

消息	方向	用途
ListToolsRequest/Result	Client → Server	询问服务器有哪些工具
CallToolRequest/Result	Client → Server	执行指定工具
ReadResourceRequest/Result	Client → Server	读取资源数据
GetPromptRequest/Result	Client → Server	获取预构建提示词

完整请求流程

用户问 "What repositories do I have?" 时的 12 步流程：

1. 用户提交问题到你的服务器
2. 服务器需要知道有哪些工具可用
3. 通过 MCP Client 向 MCP Server 发送 ListToolsRequest
4. MCP Server 返回可用工具列表
5. 服务器把用户问题 + 工具列表发送给 Claude
6. Claude 决定需要调用某个工具
7. 服务器通过 MCP Client 执行 Claude 指定的工具
8. MCP Server 调用实际的 GitHub API
9. GitHub 返回数据，通过 MCP Server 和 Client 传回
10. 服务器把工具结果发送给 Claude
11. Claude 基于数据生成最终回答
12. 服务器把回答返回给用户

"Yes, this flow involves many steps, but each component has a clear responsibility. The MCP client abstracts away the complexity of server communication."

构建 MCP 服务器

使用官方 Python SDK，通过装饰器定义工具，SDK 自动处理 JSON schema 生成。

"Instead of writing complex JSON schemas by hand, you can define tools with decorators and let the SDK handle the heavy lifting."

初始化服务器

```
from mcp.server.fastmcp import FastMCP

mcp = FastMCP("DocumentMCP", log_level="ERROR")
```

定义工具

用 `@mcp.tool` 装饰器定义工具。SDK 根据 Python 类型提示和 `Field` 描述自动生成工具 schema：

```
@mcp.tool(
    name="read_doc_contents",
    description="Read the contents of a document and return it as a string."
)
def read_document(
    doc_id: str = Field(description="Id of the document to read")
):
    if doc_id not in docs:
        raise ValueError(f"Doc with id {doc_id} not found")
    return docs[doc_id]
```

```
@mcp.tool(
    name="edit_document",
    description="Edit a document by replacing a string."
)
def edit_document(
    doc_id: str = Field(description="Id of the document"),
    old_str: str = Field(description="The text to replace. Must match exactly."),
    new_str: str = Field(description="The new text to insert.")
):
    if doc_id not in docs:
        raise ValueError(f"Doc with id {doc_id} not found")
    docs[doc_id] = docs[doc_id].replace(old_str, new_str)
```

SDK 方式的优势

- 不需要手写 JSON schema
- 类型提示提供自动验证
- Field 描述帮助 Claude 理解参数用途
- 异常处理直接用 Python 原生方式
- 工具注册通过装饰器自动完成

MCP Inspector 调试

"The Python MCP SDK includes a built-in browser-based inspector that lets you debug and test your server in real-time."

```
mcp dev mcp_server.py
```

启动后在浏览器打开 `http://127.0.0.1:6274` ，可以：

- 查看所有已注册的工具、资源、提示词
- 在界面中直接测试工具调用
- 验证参数和返回值
- 测试多工具交互序列
- 服务器状态在调用之间保持

工具定义详细字段（官方文档补充）

字段	说明
<code>name</code>	1-128 字符，区分大小写，允许 A-Z a-z 0-9 _ - .
<code>title</code>	人类可读的显示名称
<code>description</code>	工具的功能描述
<code>inputSchema</code>	JSON Schema（默认 2020-12） ， 定义参数
<code>outputSchema</code>	可选，声明返回值的 JSON Schema
<code>annotations</code>	附加元信息（如 audience、priority）
<code>taskSupport</code>	forbidden / optional / required，控制长时间运行

工具结果支持多种类型：text、image (base64)、audio (base64)、resource_link、embedded resource、structuredContent。

Resources：暴露数据

Resources 让你向客户端暴露只读数据，类似 HTTP 服务器的 GET 请求处理器。它们由应用代码控制，而非模型。

"Resources in MCP servers allow you to expose data to clients, similar to GET request handlers in a typical HTTP server."

两种 Resource 类型

Direct Resources（直接资源）

静态 URI，不需要参数：

```
@mcp.resource(
    "docs://documents",
    mime_type="application/json"
)
def list_docs() → list[str]:
    return list(docs.keys())
```

Templated Resources（模板资源）

URI 中包含参数，SDK 自动解析并传递给函数：

```
@mcp.resource(
    "docs://documents/{doc_id}",
    mime_type="text/plain"
)
def fetch_doc(doc_id: str) → str:
    if doc_id not in docs:
        raise ValueError(f"Doc with id {doc_id} not found")
    return docs[doc_id]
```

MIME 类型

类型	用途
application/json	结构化数据
text/plain	纯文本
application/pdf	二进制文件

SDK 自动序列化返回值，不需要手动转 JSON。

客户端读取 Resource

```
async def read_resource(self, uri: str) → Any:
    result = await self.session().read_resource(AnyUrl(uri))
    resource = result.contents[0]

    if isinstance(resource, types.TextResourceContents):
        if resource.mimeType == "application/json":
            return json.loads(resource.text)

    return resource.text
```

响应包含 MIME 类型，客户端据此决定如何解析内容。

实际应用：文档引用

"When a user mentions a document, your system automatically injects the document's contents into the prompt sent to Claude, eliminating the need for Claude to use tools to fetch the information."

用户输入 `@document_name` 时，系统自动将文档内容注入到 prompt 中，无需 Claude 额外调用工具。

Resource 订阅（官方文档补充）

服务器可以声明 `subscribe` 能力，客户端通过 `resources/subscribe` 监听资源变化：

- `notifications/resources/updated`：某个资源内容变化
- `notifications/resources/list_changed`：可用资源列表变化

Resource 还支持 annotations（audience、priority、lastModified）和 icons 字段。

Prompts: 预构建 workflow

Prompts 让你定义精心编写的指令模板，用户通过 slash 命令触发。比用户自己写的提示词更可靠。

"Users can already ask Claude to do most tasks directly. But they'll get much better results if you provide a thoroughly tested, specialized prompt that handles edge cases and follows best practices."

定义 Prompt

```
@mcp.prompt(
    name="format",
    description="Rewrites the document in Markdown format."
)
def format_document(
    doc_id: str = Field(description="Id of the document to format")
) → list[base.Message]:
    prompt = f"""
Your goal is to reformat a document to be written with markdown syntax.

The id of the document you need to reformat is:
<document_id>
{doc_id}
</document_id>

Add in headers, bullet points, tables, etc as necessary.
Use the 'edit_document' tool to edit the document.
"""
    return [base.UserMessage(prompt)]
```

客户端使用 Prompt

```
async def list_prompts(self) → list[types.Prompt]:
    result = await self.session().list_prompts()
    return result.prompts

async def get_prompt(self, prompt_name, args: dict[str, str]):
    result = await self.session().get_prompt(prompt_name, args)
    return result.messages
```

用户输入 `/format plan.md`，系统获取 prompt 消息列表，直接发送给 Claude 处理。

Prompt 的优势

- **一致性：**用户每次都能得到可靠结果

- **专业性**: 你可以把领域知识编码进 prompt
- **可复用**: 多个客户端应用可以使用同一套 prompts
- **可维护**: 在一处更新 prompt, 所有客户端受益

"Prompts work best when they're specialized for your MCP server's domain. The goal is to provide prompts that are so well-crafted that users prefer them over writing their own instructions."

三大原语对比

Tools、Resources、Prompts 各由不同层级控制。选对原语是设计好 MCP 系统的关键。

"The key insight is that each primitive is controlled by a different part of your application stack."

核心对比表

原语	控制者	触发方式	数据方向	适用场景
Tools	AI 模型 (Claude)	Claude 自主决定	双向 (读写)	搜索、计算、API 调用、文件操作
Resources	应用代码	App 代码触发	只读	自动补全、上下文注入、UI 数据源
Prompts	用户	用户操作 (slash/按钮)	输入到模型	格式化、审查、报告等工作流

选择判断框架

- 需要给 Claude 新能力？ → **Tools**
- 需要获取数据给 UI 或上下文？ → **Resources**
- 需要创建用户可触发的预定义工作流？ → **Prompts**

在 Claude 界面中的体现

"You can see all three primitives in action in Claude's official interface. The workflow buttons demonstrate prompts, the Google Drive integration shows resources in action, and when Claude executes code, it's using tools."

- 工作流按钮（聊天框下方） = Prompts 的体现
- "Add from Google Drive" 功能 = Resources 的体现
- Claude 执行代码和计算 = Tools 的体现

MCP Server 能力声明（官方文档补充）

能力	声明	可选特性
Tools	<code>capabilities.tools</code>	<code>listChanged</code> 工具列表变化通知
Resources	<code>capabilities.resources</code>	<code>subscribe</code> 订阅变化、 <code>listChanged</code>
Prompts	<code>capabilities.prompts</code>	<code>listChanged</code> 提示词列表变化通知

客户端也有自己的原语：**Sampling**（让服务器请求 LLM 生成）、**Elicitation**（请求用户输入）、**Logging**（结构化日志）。

| 实战要点与考试准备

课程覆盖了 MCP 的核心概念和实战技能。以下是最值得记住的要点。

开发 Checklist

1. 用 FastMCP 初始化: `mcp = FastMCP("name")`
2. 用装饰器定义工具: `@mcp.tool` , Field 描述每个参数
3. 用 Inspector 测试: `mcp dev server.py` , 不需要写测试脚本
4. 实现客户端两个核心函数: `list_tools()` 和 `call_tool()`
5. Resource 用 URI 标识: 直接 URI 或模板 URI
6. Prompt 返回消息列表: 可包含多轮 user/assistant 消息

常见误区

- ❌ 把 MCP 当成 Tool Use 的替代 (它们互补)
- ❌ 所有数据都用 Tools 获取 (只读数据用 Resources 更高效)
- ❌ 忽略 MIME 类型 (影响客户端如何解析响应)
- ❌ 不测试就部署 (Inspector 是为此而生的)

考试相关要点

考点	关键答案
MCP 的核心类比	USB-C，统一的 AI 应用连接标准
三个角色	Host (AI App) → Client (Bridge) → Server (Tools)
两种传输	stdio (本地) / Streamable HTTP (远程)
Tools 的控制者	模型 (Claude 自主决定调用)
Resources 的控制者	应用代码 (App 决定获取)
Prompts 的控制者	用户 (用户触发 workflows)
MCP vs Tool Use	互补：MCP 提供工具定义，Tool Use 是调用机制
FastMCP 的优势	装饰器 + 类型提示 → 自动生成 JSON schema
MCP Inspector 命令	<code>mcp dev server.py</code>
Resource URI 类型	Direct (静态) vs Templated (含参数)

organized by Syneira

基于 Anthropic CCAF 认证课程 "Introduction to Model Context Protocol" (14 lessons) 整理
含 modelcontextprotocol.io 官方文档补充 (Architecture, Transports, Resources, Tools, Prompts)
课程完成度 100% • 2026.06